

Reproduccion alternativa de gradient pathologies en PINNs

Difusion de calor 1D en una barra metalica

Trabajo practico de Optimizacion

Mayo 2026

Abstract

Se replica el nucleo metodologico de Wang, Teng y Perdikaris (*Understanding and mitigating gradient pathologies in physics-informed neural networks*) sobre un problema distinto: difusion de calor unidimensional en una barra metalica. Se comparan una PINN clasica (M1), una PINN con balance adaptativo de terminos de perdida basado en estadisticas de gradientes (M2) y la solucion analitica del problema. No se implementa la arquitectura nueva del paper. Tambien se incluye busqueda bayesiana de hiperparametros y una comparacion de optimizadores.

1 Problema fisico

La barra tiene longitud $L = 1$ m, difusividad termica $\alpha = 10^{-4}$ m²/s, extremos mantenidos a temperatura ambiente $T_{\text{amb}} = 20^\circ\text{C}$ y escala de temperatura $\Delta T = 80^\circ\text{C}$. La ecuacion dimensional es

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial X^2}, \quad 0 < X < L, \quad t > 0. \quad (1)$$

Las condiciones de borde son

$$T(0, t) = T(L, t) = T_{\text{amb}}. \quad (2)$$

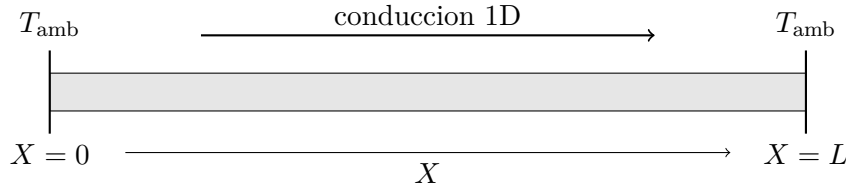


Figure 1: Sistema de referencia para la barra metalica.

Se adimensionaliza con

$$x = \frac{X}{L}, \quad \tau = \frac{\alpha t}{L^2}, \quad u = \frac{T - T_{\text{amb}}}{\Delta T}. \quad (3)$$

El problema usado por la PINN es entonces

$$u_\tau - u_{xx} = 0, \quad 0 < x < 1, \quad 0 < \tau < \tau_f, \quad (4)$$

$$u(0, \tau) = u(1, \tau) = 0, \quad (5)$$

$$u(x, 0) = u_0(x), \quad (6)$$

donde $\tau_f = 0.25$, equivalente a $t_f = 2500$ s.

La condicion inicial es una suma de modos compatibles con los bordes:

$$u_0(x) = \sin(\pi x) + 0.5 \sin(3\pi x) + 0.25 \sin(5\pi x). \quad (7)$$

La solucion analitica por separacion de variables es

$$u(x, \tau) = \sum_{n \in \{1,3,5\}} a_n \sin(n\pi x) \exp(-(n\pi)^2 \tau), \quad (8)$$

con $(a_1, a_3, a_5) = (1, 0.5, 0.25)$.

2 Datos generados para la PINN

No se usa un dataset fijo de interior. En cada iteracion se generan nuevos minibatches:

$$x_r \sim \mathcal{U}(0, 1), \quad \tau_r \sim \mathcal{U}(0, \tau_f) \quad \text{residuo}, \quad (9)$$

$$x_0 \sim \mathcal{U}(0, 1), \quad \tau_0 = 0 \quad \text{condicion inicial}, \quad (10)$$

$$\tau_b \sim \mathcal{U}(0, \tau_f), \quad x_b \in \{0, 1\} \quad \text{bordes}. \quad (11)$$

En la corrida final se usa batch 256 para residuo, condicion inicial y borde. La mitad del batch de borde esta en $x = 0$ y la otra mitad en $x = 1$.

La grilla de evaluacion es regular, con $n_x = n_\tau = 180$. El error reportado es

$$\varepsilon_{L^2} = \frac{\|u_\theta - u_{\text{exacta}}\|_2}{\|u_{\text{exacta}}\|_2}. \quad (12)$$

3 PINN y perdidas

La red $u_\theta(x, \tau)$ es una MLP totalmente conectada con activacion tanh. Antes del primer layer se escala la entrada a $[-1, 1]^2$:

$$z = 2 \frac{[x, \tau] - [0, 0]}{[1, \tau_f] - [0, 0]} - 1. \quad (13)$$

La inicializacion es Xavier normal para pesos y cero para sesgos. La arquitectura principal final es

$$[2, 50, 50, 50, 1]. \quad (14)$$

Las perdidas son

$$L_{\text{res}} = \text{mean}((u_\tau - u_{xx})^2), \quad (15)$$

$$L_{\text{ic}} = \text{mean}((u_\theta(x, 0) - u_0(x))^2), \quad (16)$$

$$L_{\text{bc}} = \text{mean}(u_\theta(0, \tau)^2 + u_\theta(1, \tau)^2). \quad (17)$$

Las derivadas u_τ y u_{xx} se calculan con autograd de PyTorch.

4 Modelos comparados

El baseline M1 minimiza

$$L_{\text{M1}} = L_{\text{res}} + L_{\text{ic}} + L_{\text{bc}}. \quad (18)$$

El modelo M2 usa la correccion de balance adaptativo del paper:

$$L_{\text{M2}} = L_{\text{res}} + \lambda_{\text{ic}} L_{\text{ic}} + \lambda_{\text{bc}} L_{\text{bc}}. \quad (19)$$

Cada 10 iteraciones se estiman pesos instantaneos

$$\hat{\lambda}_i = \frac{\max_\theta |\nabla_\theta L_{\text{res}}|}{\text{mean}_\theta |\nabla_\theta L_i|}, \quad i \in \{\text{ic}, \text{bc}\}, \quad (20)$$

y se actualizan con media movil

$$\lambda_i \leftarrow \beta \lambda_i + (1 - \beta) \hat{\lambda}_i. \quad (21)$$

En la corrida final se usa $\beta = 0.9$. Las estadisticas se computan sobre los pesos de las capas lineales, como en el repositorio de los autores.

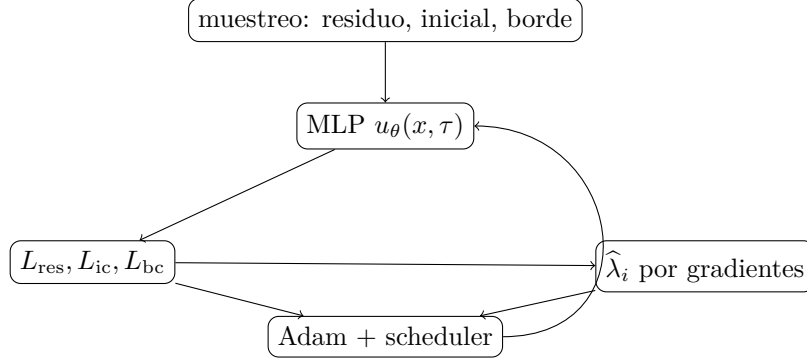


Figure 2: Flujo de entrenamiento para M2. M1 usa el mismo flujo sin actualizar pesos adaptativos.

5 Optimizacion e hiperparametros

La corrida principal usa Adam con learning rate inicial 10^{-3} , sin weight decay, semilla 0, CPU y 16 threads. El scheduler es exponencial:

$$\eta_k = \eta_0 \left(0.9^{1/1000}\right)^k. \quad (22)$$

El entrenamiento principal dura 3000 iteraciones.

La busqueda bayesiana usa Optuna/TPE con 8 trials y 800 iteraciones por trial. El espacio explorado fue:

Parametro	Rango
ancho	$\{24, 32, 48, 64, 80\}$
profundidad	enteros de 2 a 5
learning rate	log-uniforme en $[10^{-4}, 3 \cdot 10^{-3}]$
batch	$\{128, 256, 384\}$
β adaptativo	uniforme en $[0.75, 0.97]$

6 Resultados

Modelo	error L^2 rel.	tiempo [s]	λ_{ic}	λ_{bc}
M1	0.293134	134.47	1.00	1.00
M2	0.102912	421.23	219148.36	276036.68

Table 1: Comparacion principal con arquitectura $[2, 50, 50, 50, 1]$ y 3000 iteraciones.

La mejora de M1 a M2 fue $2.85\times$ en error relativo L^2 .

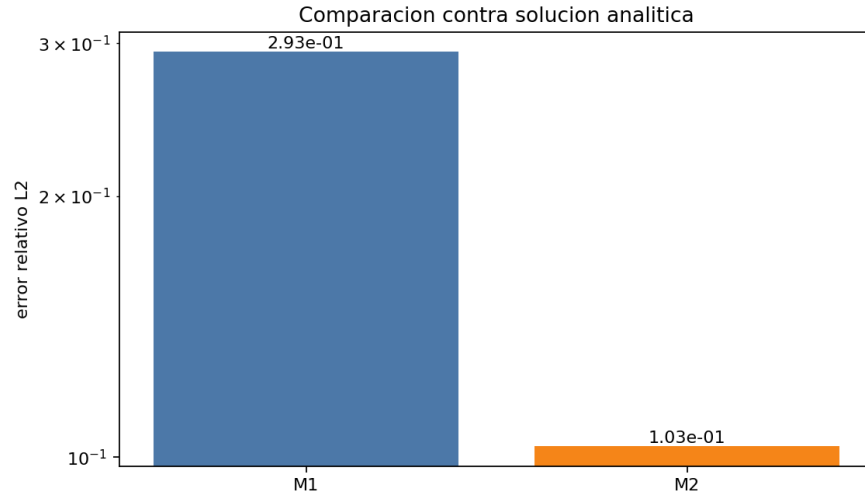


Figure 3: Error relativo L^2 contra solucion analitica.

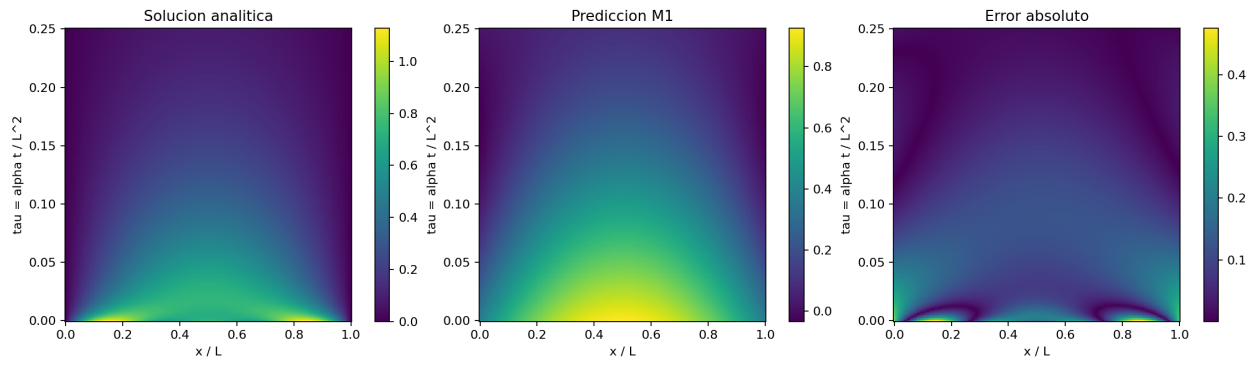


Figure 4: Solucion exacta, prediccion M1 y error absoluto.

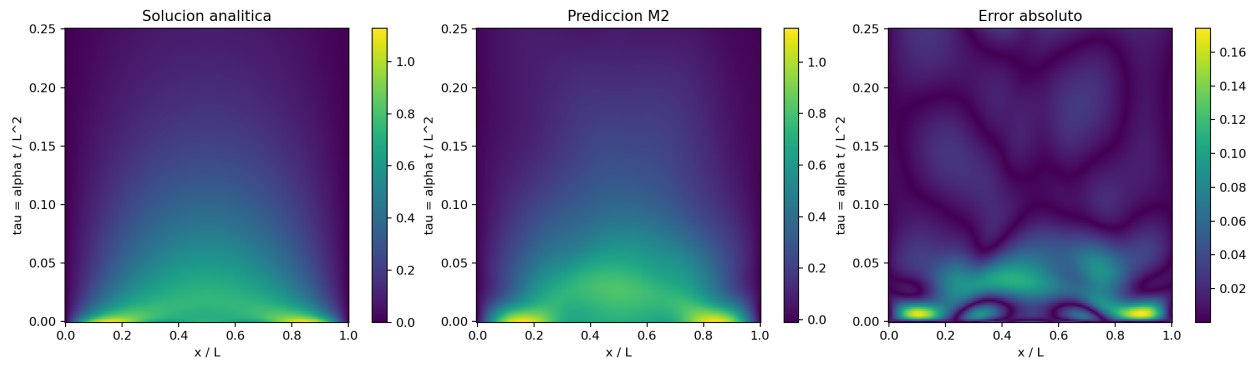


Figure 5: Solucion exacta, prediccion M2 y error absoluto.

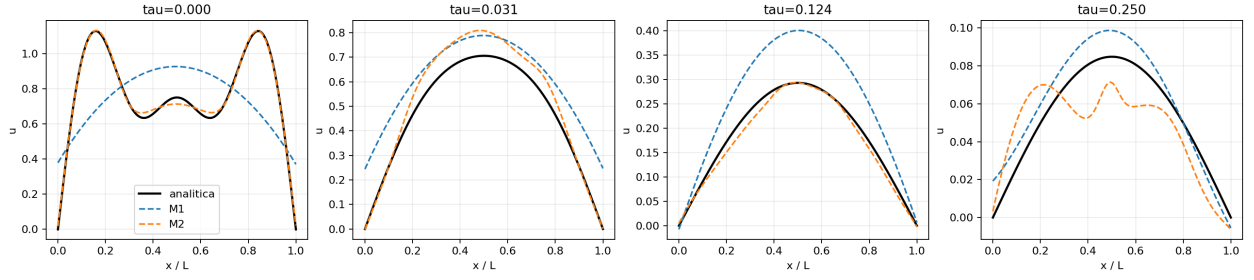


Figure 6: Cortes espaciales en tiempos adimensionales fijos.

7 Gradientes y sensibilidad

Los histogramas de gradientes reproducen el análisis cualitativo del paper: se inspeccionan los gradientes de L_{res} , L_{ic} y L_{bc} por capa para detectar desbalance entre terminos.

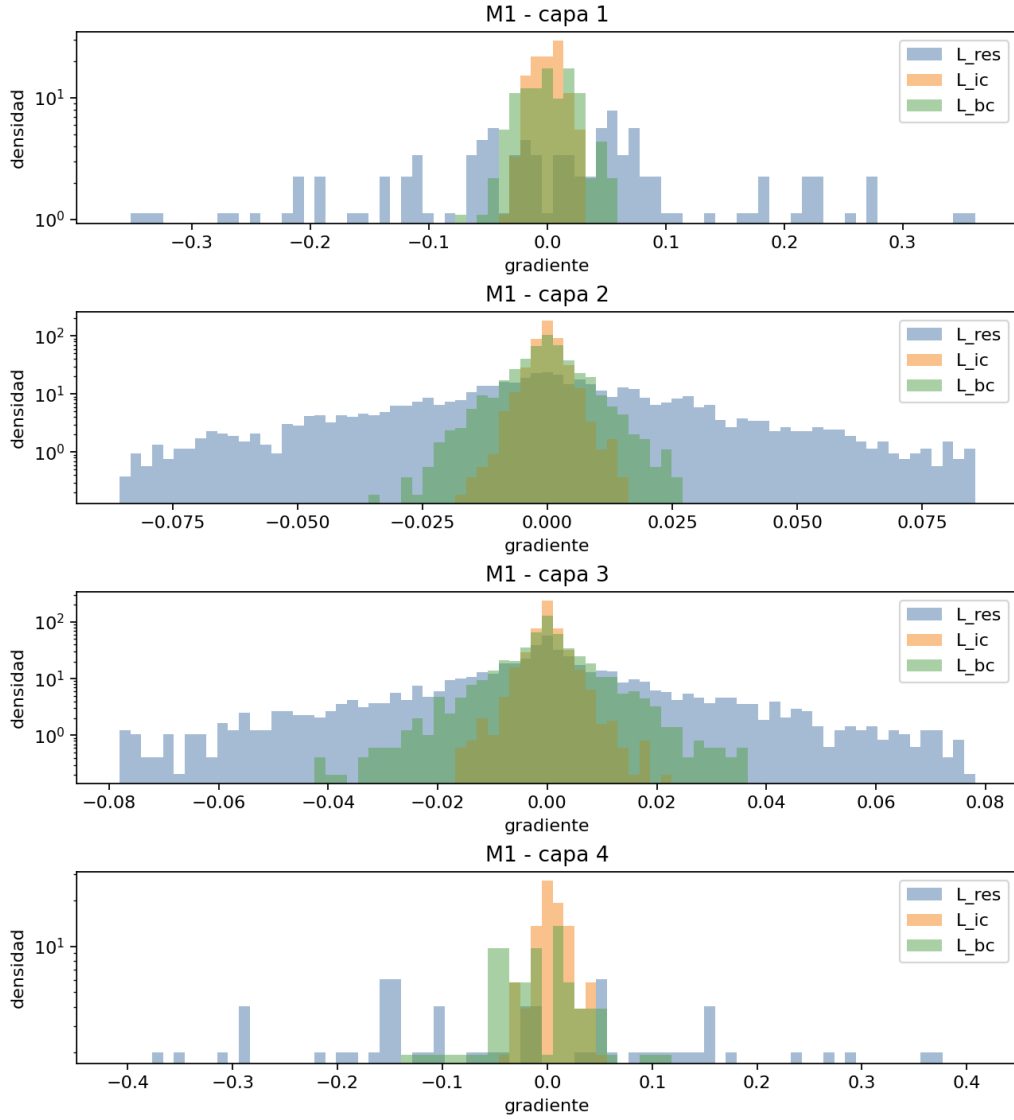


Figure 7: Histogramas de gradientes por capa para M1.

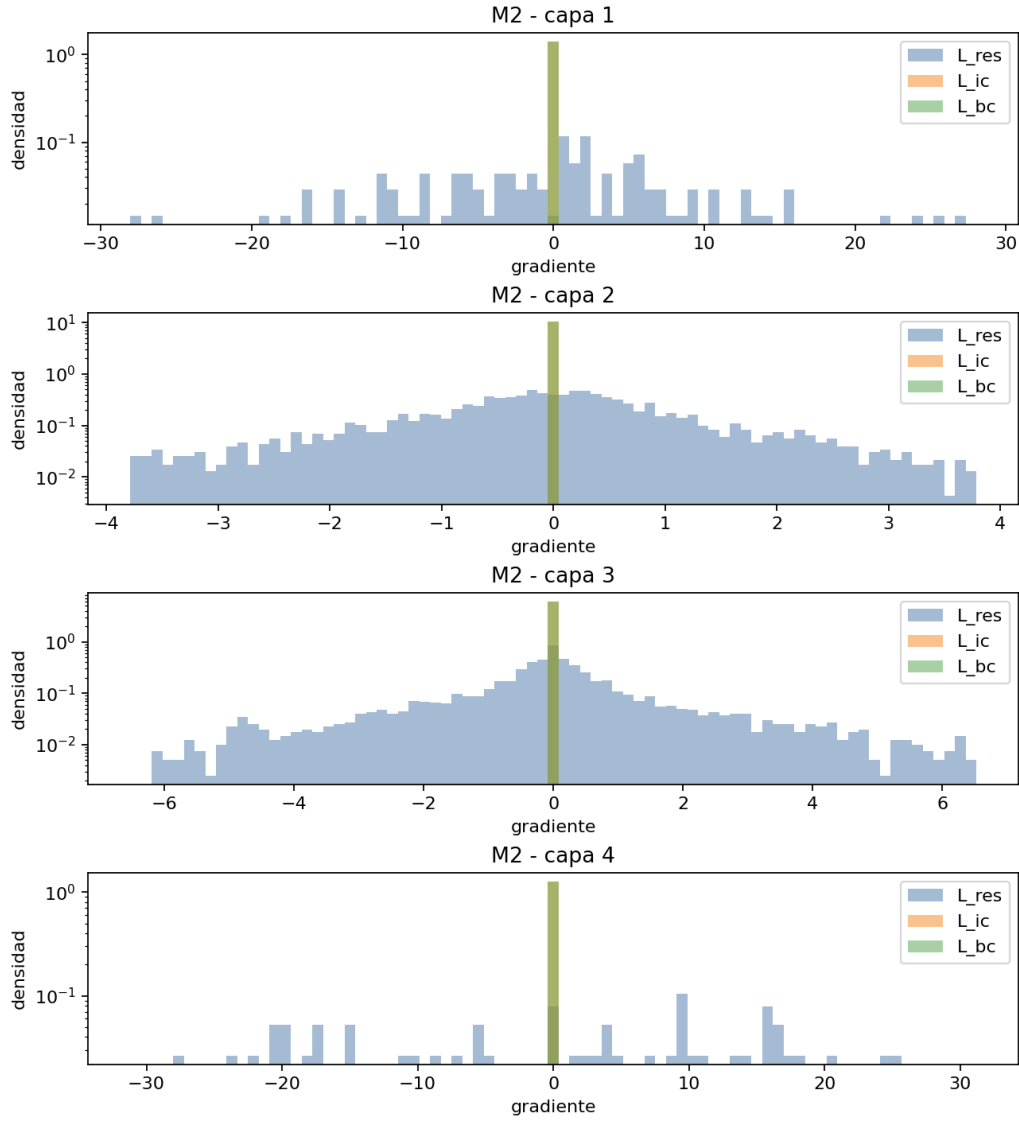


Figure 8: Histogramas de gradientes por capa para M2.

El mejor trial corto de la búsqueda bayesiana tuvo $\text{width} = 48$, $\text{depth} = 5$, $\eta = 0.0010166$, $\text{batch} = 384$ y $\beta = 0.76325$, con error corto 0.14138. Sin embargo, al validarlo por 3000 iteraciones, los pesos adaptativos crecieron hasta orden 10^6 y el error final quedo en 0.18175 para M2. El regimen conservador con $\beta = 0.9$ obtuvo 0.10291. Por eso la seleccion final no se baso solo en el mejor trial corto, sino en validacion de estabilidad.

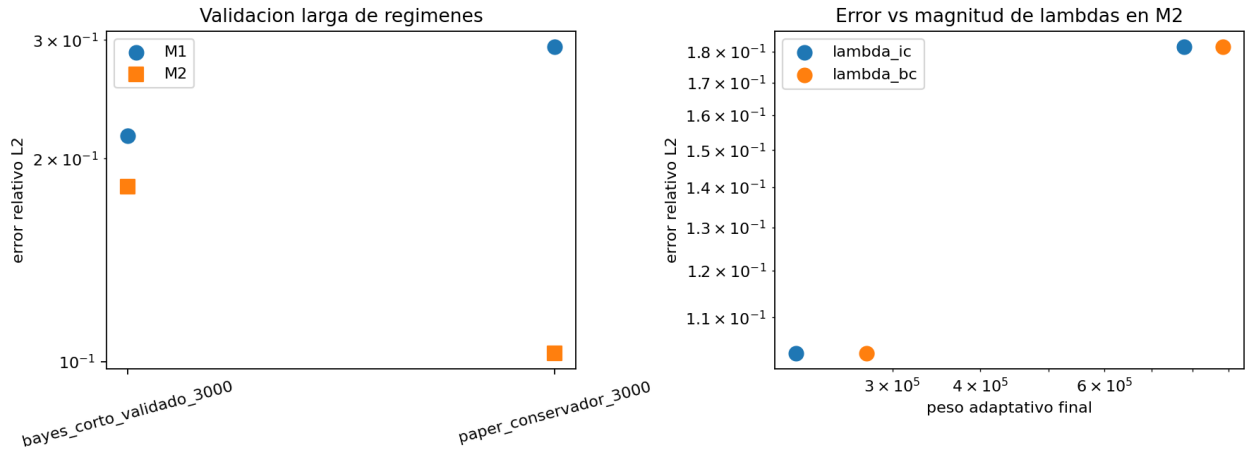


Figure 9: Sensibilidad entre el mejor regimen bayesiano corto validado a 3000 iteraciones y el regimen conservador final.

8 Búsqueda bayesiana y optimizadores

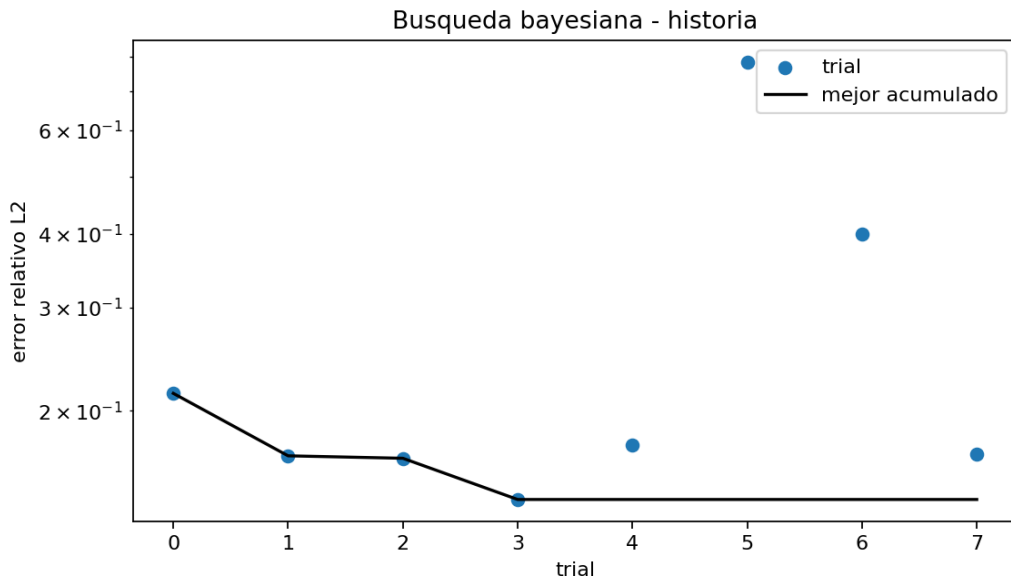


Figure 10: Historia de trials y mejor error acumulado en la búsqueda bayesiana.

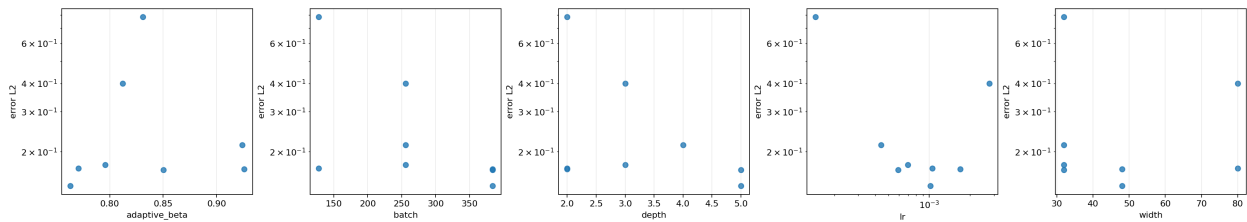


Figure 11: Error contra parametros explorados por Optuna/TPE.

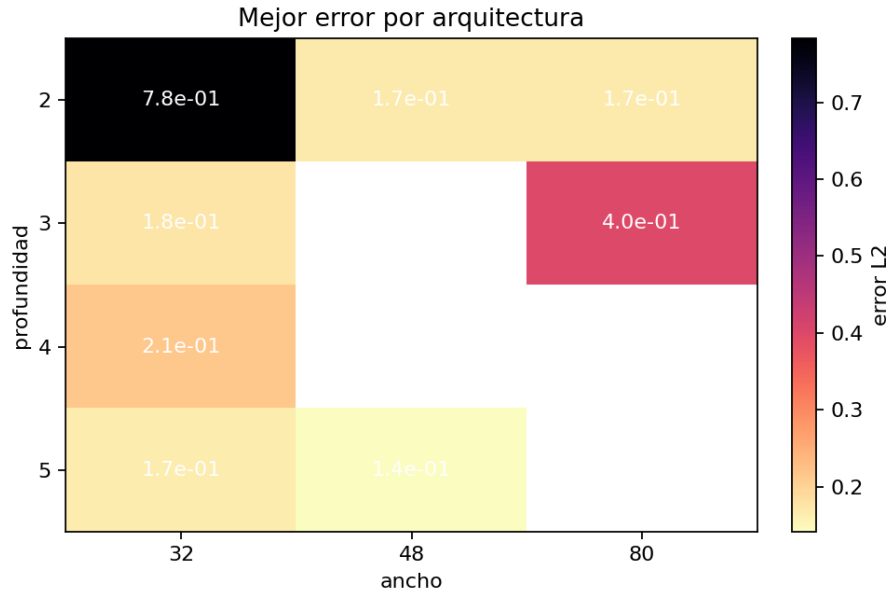


Figure 12: Mejor error observado por ancho y profundidad.

La comparacion de optimizadores se hizo con presupuesto fijo y M2:

Optimizador	error L^2 rel.	tiempo [s]
Adam + LBFGS(10)	0.166899	109.02
Adam	0.176340	122.15
AdamW	0.176340	71.27
SGD	NaN	116.69

Table 2: Comparacion de optimizadores con 1000 iteraciones, batch 128 y arquitectura [2, 32, 32, 32, 1].

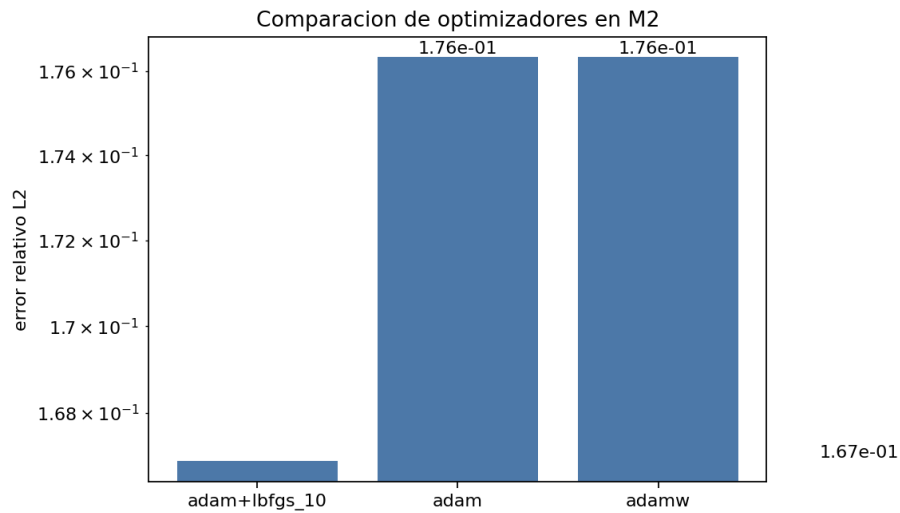


Figure 13: Comparacion grafica de optimizadores. SGD divergio con el learning rate comun usado en esta prueba.

9 Reproducibilidad

Los comandos principales fueron:


```
conda env create -f environment.yml
conda activate tp-optimizacion-ger

python experimento.py bayes --trials 8 --search-iters 800 --threads 16

python experimento.py todos \
  --iters 3000 --batch 256 --width 50 --depth 3 \
  --lr 0.001 --adaptive-beta 0.9 \
  --log-every 500 --nx 180 --nt 180 --threads 16

python experimento.py optimizadores \
  --iters 1000 --batch 128 --width 32 --depth 3 \
  --lr 0.001 --adaptive-beta 0.9 \
  --log-every 500 --nx 120 --nt 120 \
  --lbfgs-steps 10 --threads 16

python analisis_sensibilidad.py
tectonic reporte.tex
```